Introduction to Web World

Building blocks

- Web Server
- Web Browser
- HTTP/HTTPS
- CGI programs (out-dated technology)
- Application Server
- HTML/Javascript
- Database
- XML data

Web Server

- A Web Server is a program running on the server that listens for incoming *HTTP* requests and serves those requests as they come in.
- Once the web server receives a request, depending on the type of request the web server might look for a web page, or it might execute a program on the server.
- It will always return some kind of results to the web browser, even if it is simply an error message saying that it couldn't process the request.
- By default the role of a web server is to serve static web pages using the http protocol.
- Web servers can be made dynamic by adding additional processing capability to the server

Web Server (Cont'd)

- By default the Web Servers listen on Port 80 and in test environments it could be 8080.
- When a request comes into the Web server, the Web server simply passes the request to the program best able to handle it. The Web server doesn't provide any functionality beyond simply providing an environment in which the server-side program can execute and pass back the generated responses.
- It may employ various strategies for fault tolerance and scalability such as load balancing, caching, and clustering.

Static content

Request file

Retrieve HTML file, images,

Send file

Dynamic pages

Request service

Do Computation

Generate HTML page with results of computation

Return dynamically generated HTML file

Extensions of Web Server

- Several different tools are available for extending the server capabilities
  - Apache Tomcat
  - VB .Net architecture
  - IBM Websphere
  - GlassFish
  - CGI scripting

  } Application Servers

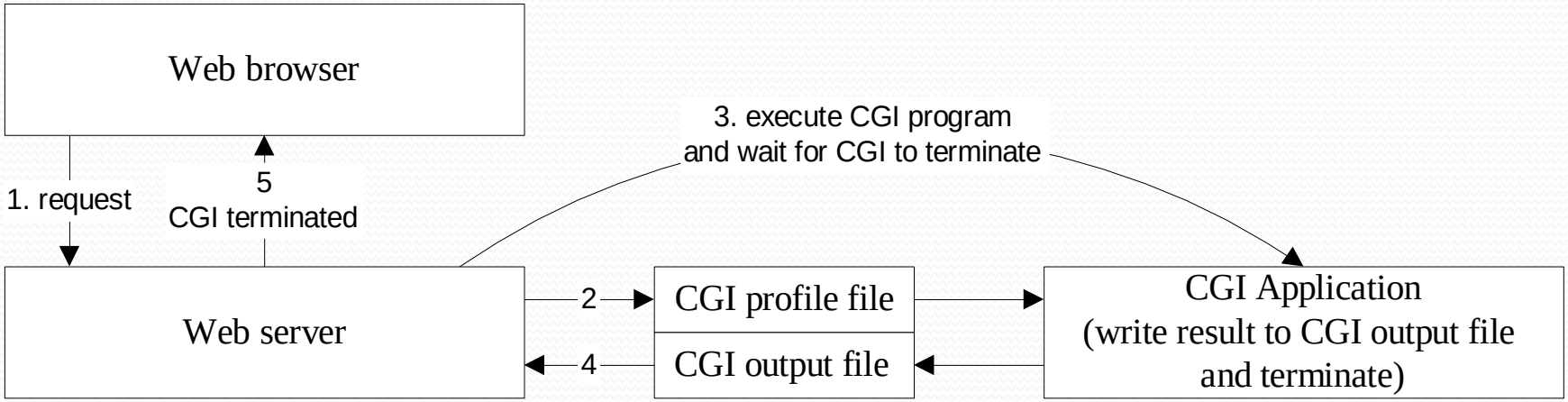
- These tools process incoming requests from the user and generate custom html pages.

CGI – Common Gateway Interface

- CGI is a specification that was introduced in 1993 by NCSA for HTTP based servers.
  - Client requests program to be run on server-side
  - Web server passes parameters to program through UNIX shell environment variables
  - Program spawned as separate process via fork
  - Program's output => Results
  - Server passes back results (usually in form of HTML)

CGI – Traditional approach

CGI – Common Gateway Interface



1. Web browser request a response from Web server.
2. Web server create CGI profile file that contains information about CGI variable, server and CGI output file.
3. Web server starts CGI application and wait for its termination.
4. CGI program runs and writes the result to CGI output file and then terminates.
5. Web server reads from CGI output file and send back the response to Web browser.

How a web server works

- A web server is technology that hosts a website's code and data. When you enter a URL in your browser, the URL is actually the address identifier of the web server.

Your browser and web server communicate as follows:

- The browser uses the URL to find the server's IP address
- The browser sends an HTTP request for information
- The web server communicates with a database server to find the relevant data
- The web server returns static content such as HTML pages, images, videos, or files in an HTTP response to the browser
- The browser then displays the information to you

Web and App Server examples

Web Server	Programming Environment
Apache	PHP, CGI
IIS	ASP (.NET)
Tomcat	Servlets
Jetty	Servlets

Application Server	Programming Environment
WAS (IBM's WebSphere)	EJB, JMS
WebLogic (Oracle's)	EJB, JMS
Jboss	EJB,
MTS	COM+

Application Server

- This is an extension of Web server and replacement of CGI programs. Sometimes, Application Server itself has Web Server inside.
- An application server extends the capabilities of a web server by supporting dynamic content generation, application logic, and integration with various resources. It provides a runtime environment where you can run application code and interact with other software components, like messaging systems and databases. It uses business logic to transform data more meaningfully than a web server..
- The application server manages its own resources such as gate-keeping duties include security, transaction processing, resource pooling, database connectivity and messaging. Like a Web server, an application server may also employ various scalability and fault-tolerance techniques.

How an application server works

- When you attempt to access interactive content on a website, the process works as follows:
- The browser uses the URL to finds the server's IP address
- The browser sends an HTTP request for information
- The web server transfers the request to the application server
- The application server applies business logic and communicates with other servers and third-party systems to fulfill the request
- The application server renders a new HTML page and returns it as a response to the web server
- The web server returns the response to the browser
- The browser displays the information to you

Core Functions and Features of Application Server

- **Business Logic Execution**: Processes complex, dynamic, data-driven application requests, acting as a, backend to web servers.
- **Database Interaction & Pooling**: Manages, stores, and retrieves data from databases, often using connection pools to improve efficiency.
- **Session Management:** Maintains user-specific data and state across interactions, critical for user authentication and workflows.
- **Transaction & Security Management:** Ensures secure, atomic transactions and provides user authentication, authorization, and data encryption.
- **Performance & Scalability:** Utilizes caching and load balancing (clustering) to handle high-volume user traffic.
- **Messaging & Integration:** Manages, asynchronous communication between different components, such as IBM-MQ, JMS, point-to-point, publish-subscribe.

HTTP – Application Layer Protocol

- Hypertext Transfer Protocol
- User applications implement this protocol
- Different applications use different protocols
  - Web Servers/Browsers use HTTP
  - File Transfer Utilities use FTP
  - Electronic Mail applications use SMTP
  - Network elements use SNMP
- Interacts with transport layer(TCP/IP) to send messages
- Lightweight protocol for the web involving a single request & response for communication
- Default designated HTTP port is 80

HTTP - Protocol

- HTTP is a stateless protocol
  - Request/Response occurs across a single network connection
  - At the end of the exchange the connection is closed
  - This is required to make the server more scalable
- Web Sites maintain persistent authentication so user does not have to authenticate repeatedly
- While using HTTP persistent authentication is maintained using a token exchange mechanism
- HTTP 1.1 has a special feature (keep-alive) which allows clients to use same connection over multiple requests
  - Not many servers support this
  - Requests have to be in quick succession

HTTP - messages

- Each message, whether a request or a response, has three parts:
  1. The request or the response line
  2. A header section
  3. The body of the message
- The first part of the message is the request line, containing:
  - A method (HTTP command) such as GET or POST
  - A document address, and
  - An HTTP version number
- Example:
  - GET /index.html HTTP/1.0

HTTP – Request methods

Provides 8 request methods

- Get: Used to request data from server
  (By convention **get** will not change data on server)
- Post: Used to post data to the server
- Head: returns just the HTTP headers for a resource.
- Put: allows you to "put" (upload) a resource (file) on to a web server so that it be found under a specified URI.
- Delete: allows you to delete a resource (file).
- Connect:
- Options: To determine the type of requests server will handle
- Trace: Debugging

HTTP – request header

- The second part of a request is optional header information, such as:
  - What the client software is
  - What formats it can accept
- All information is in the form Name: Value
- Example:

```
User-Agent: Mozilla/2.02Gold (WinNT; I)
Accept: image/gif, image/jpeg, */*
```

- A *blank line* ends the header

HTTP – request header

- Accept: *type/subtype, type/subtype, …*
  - Specifies media types that the client prefers to accept
- Accept-Language: en, fr, de
  - Preferred language (For example: English, French, German)
- User-Agent: *string*
  - The browser or other client program sending the request
- From: dave@acm.org
  - Email address of user of client program
- Cookie: *name=value*
  - Information about a cookie for that URL
  - Multiple cookies can be separated by commas

HTTP – request body

- The third part of a request (after the blank line) is the entity-body, which contains optional data
  - The entity-body part is used mostly by POST requests
  - The entity-body part is always empty for a GET request

HTTP – GET and POST

- GET and POST allow information to be sent back to the web server from a browser
  - e.g. when you click on the “submit” button of a form the data in the form is send to the server, as "name=value" pairs.
- Choosing GET as the "method" will append all of the data to the URL and it will show up in the URL bar of your browser.
  - The amount of information you can send back using a GET is restricted as URLs can only be 1024 characters.
- A POST sends the information in HTTP request body back to the web server and it won't show up in the URL bar.
  - This allows a lot more information to be sent to the server
  - The data sent back is not restricted to textual data and it is possible to send files and binary data such as serialized Java objects.

HTTP – response

- The server response is also in three parts
- The first part is the status line, which tells:
  - The HTTP version
  - A status code
  - A short description of what the status code means
- Example: HTTP/1.1 404 Not Found
- Status codes are in groups:

100-199	Informational
200-299	The request was successful
300-399	The request was redirected
400-499	The request failed
500-599	A server error occurred

HTTP – Status codes

200 OK

Everything worked, here's the data

301 Moved Permanently

URI was moved, but here's the new address for your records

302 Moved temporarily

URL temporarily out of service, keep the old one but use this one for now

400 Bad Request

There is a syntax error in your request

401Unauthorized access

403 Forbidden

You can't do this, and we won't tell you why

404 Not Found

No such document

405 Method not allowed

408 Request Time-out, 504 Gateway Time-out

Request took too long to fulfill for some reason

HTTP – Response header

The second part of the response is header information, ended by a blank line

Example:

```
Content-Length: 2532
Connection: Close
Server: GWS/2.0
Date: Sun, 01 Dec 2002 21:24:50 GMT
Content-Type: text/html
Cache-control: private
Set-Cookie:
PREF=ID=05302a93093ec661:TM=1038777890:LM=1038
777890:S=yNWNjraftUz299RH; expires=Sun, 17-Jan-
2038 19:14:07 GMT; path=/; domain=.google.com
```

All on one line

HTTP – Response header

Example:

Status line:

HTTP/1.1 200 OK

Response headers:

Date: Wed, 10 Sep 2003 00:26:53 GMT

Server: Apache/1.3.26 (Unix) PHP/4.2.2 mod_perl/1.27 mod_ssl/2.8.10 OpenSSL/0.9.6e

Last-Modified: Tue, 09 Sep 2003 19:24:50 GMT

ETag: "1c1ad5-1654-3f5e2902”

Accept-Ranges: bytes

Content-Length: 5716

Keep-Alive: timeout=15, max=100

Connection: Keep-Alive

Content-Type: text/html

HTTP – Response body

- The third part of a server response it the entity body
- This is often an HTML page that is displayed by the browser
  - But it can also be a jpeg, a gif, plain text, etc.--anything the browser (or other client) is prepared to accept

Setting up Tomcat Server

- Pulling the tar from shared directory and un-tarring it.
- Setting up JAVA_HOME environment variable.
- Start the tomcat server by running *startup.sh* located under <Tomcat_installation_dir>\bin
  - Test the tomcat server by going to http://localhost:8080
- Setting up the port number to 80 in *server.xml* file.
  - Test the tomcat server by going to http://localhost/
- Setting up user accounts for "Tomcat Web Application Manager"
- Adding user and role in *tomcat-users.xml* file:

```
<role rolename="manager-gui"/>
<role rolename="manager-script"/>
<user username="test" password="test" roles="manager-gui"/>
```

Test Manager web page with http://localhost/manager/html/